■ PySpark Interview Cheat Sheet — Top 10 Questions & Answers

■ Q1. What is PySpark & how's it different from Apache Spark?

Apache Spark is a distributed computing framework for big data processing.
PySpark is Spark's Python API, allowing you to write Spark applications in Python instead of Scala/Java.
Difference: Spark is the core engine, PySpark is a wrapper for Python developers.

■ Q2. How do you initialize a SparkSession in PySpark?

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()
```

SparkSession is the entry point for DataFrame and SQL functionalities in PySpark.

■ Q3. What are RDDs & how do you create them?

RDD (Resilient Distributed Dataset): The fundamental data structure of Spark — immutable, distributed collection
Ways to create:
1. From an existing collection:
   rdd = spark.sparkContext.parallelize([1, 2, 3, 4])
2. From external data:
   rdd = spark.sparkContext.textFile("file.txt")

■ Q4. Transformations vs. Actions – what's the difference?

Transformations: Lazy operations that define a new RDD/DataFrame (e.g., map, filter, select).
Actions: Trigger computation and return results (e.g., count, collect, show).

Example:

```
df.filter(df.age > 30)   # Transformation
df.count()               # Action
```

■ Q5. What's a DataFrame & why is it preferred over RDD?

DataFrame: Distributed collection of data organized in named columns (like a table in SQL).
Advantages over RDDs:
- Optimized by Spark's Catalyst optimizer.
- Easier API for SQL-like queries.
- Better performance due to Tungsten execution engine.

■ Q6. How to perform basic DataFrame operations in PySpark?

```
df.show()              # Display data
df.select("name").show()
df.filter(df.age > 30).show()
df.groupBy("dept").count().show()
```

Common ops: select, filter, groupBy, agg, join, orderBy.

■ Q7. What is SparkSQL & how can you query DataFrames with it?

SparkSQL: A Spark module for running SQL queries on structured data.
You can register DataFrames as temporary views and query them with SQL:

```
df.createOrReplaceTempView("people")
spark.sql("SELECT name, age FROM people WHERE age > 30").show()
```

■ Q8. How do you read/write CSV, JSON, and Parquet in PySpark?

```
# Read
df_csv = spark.read.csv("data.csv", header=True, inferSchema=True)
df_json = spark.read.json("data.json")
df_parquet = spark.read.parquet("data.parquet")

# Write
df_csv.write.csv("out.csv", header=True)
df_json.write.json("out.json")
df_parquet.write.parquet("out.parquet")
```

■ Q9. What is Lazy Evaluation & why does it matter?

In Spark, transformations are not executed immediately. They’re recorded in a DAG (Directed Acyclic Graph).
Execution happens only when an action is called.
Benefits:
- Optimizes the execution plan.
- Avoids unnecessary computations.

■ Q10. Caching vs. Persistence – when to use which?

cache(): Stores DataFrame/RDD in memory only.
persist(): More flexible — allows storage in memory, disk, or both (with different storage levels).
Use when data is reused multiple times in iterative algorithms (e.g., ML).

Example:

```
df.cache()
df.persist(StorageLevel.MEMORY_AND_DISK)
```